

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA IN INGEGNERIA INFORMATICA
CORSO DI INGEGNERIA DEL SOFTWARE
PROF. A.R. FASOLINO - A.A. 2025-26...



**PROGETTO
DI INGEGNERIA DEL SOFTWARE**

***“SISTEMA SOFTWARE PER PRENOTAZIONI DI
POSTAZIONI IN BIBLIOTECA
UNIVERSITARIA”***

INDICE

1. SPECIFICHE INFORMALI	1
2. ANALISI E SPECIFICA DEI REQUISITI	3
2.1 ANALISI NOMI-VERBI	3
2.2 REVISIONE DEI REQUISITI	5
2.3 GLOSSARIO DEI TERMINI	6
2.4 CLASSIFICAZIONE DEI REQUISITI	7
2.4.1 Requisiti funzionali.....	7
2.4.2 Requisiti sui dati.....	8
2.4.3 Vincoli / Altri requisiti	8
2.5 MODELLAZIONE DEI CASI D'USO	10
2.5.1 Attori e casi d'uso.....	10
2.5.2 Diagramma dei casi d'uso.....	11
2.5.3 Scenari.....	11
2.6 DIAGRAMMA DELLE CLASSI.....	12
2.6.1 Responsabilità	<i>Errore. Il segnalibro non è definito.</i>
2.7 DIAGRAMMI DI SEQUENZA	14
2.7.1 Registrazione	<i>Errore. Il segnalibro non è definito.</i>
2.7.2 Accesso.....	14
2.7.3 AggiungiAuto	14
3. PIANO DI TEST FUNZIONALE	15
3.1 REGISTRAZIONE.....	15
4. PROGETTAZIONE	17
4.1 DIAGRAMMA DELLE CLASSI.....	17
4.1.1 Traduzione classi ed associazioni.....	17
4.1.2 Pattern BCED	17
4.1.2.1 Package Boundary	17
4.1.2.2 Package Controller	17
4.1.2.3 Package Entity	18
4.1.2.4 Package Database.....	18
4.2 DIAGRAMMI DI SEQUENZA	19
4.2.1 Registrazione	19
4.2.2 Accesso.....	<i>Errore. Il segnalibro non è definito.</i>
5. IMPLEMENTAZIONE.....	20
5.1 PACKAGE DATABASE	20
5.2 PACKAGE ENTITY	20
5.3 PACKAGE CONTROLLER.....	20
5.4 PACKAGE BOUNDARY	20
5.5 PACKAGE DTO	20
5.6 DIAGRAMMA DI DEPLOYMENT	21
6. TESTING	22
6.1 TEST STRUTTURALE.....	22
6.1.1 Complessità ciclomatica	22
6.1.1.1 inserisciAutoModifiche – GestioneParcoAuto.....	22
6.2 JUNIT – TEST DI UNITÀ.....	25

6.3 TEST FUNZIONALE.....26

1. Specifiche informali

Riportare la Traccia assegnata

Si desidera sviluppare un sistema software per la gestione della prenotazione di postazioni di studio all'interno di una biblioteca universitaria. Il sistema dovrà consentire agli studenti di prenotare una postazione in specifiche fasce orarie, visualizzare le proprie prenotazioni e accedere ai servizi della biblioteca in modo organizzato. Il sistema coinvolge due tipologie di utenti: studenti e bibliotecari, ciascuna con funzionalità e permessi specifici.

Il sistema consente la registrazione di utenti tramite autenticazione con credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra studente o bibliotecario, oltre a fornire nome, cognome, email istituzionale e numero di matricola o codice identificativo interno.

Ogni bibliotecario ha la possibilità di creare e gestire una o più sale studio. Per ciascuna sala, mediante apposita interfaccia grafica, è possibile specificare un nome, una descrizione, il numero totale di postazioni disponibili e gli orari di apertura giornalieri. Ogni sala può essere suddivisa opzionalmente in aree differenti, ad esempio area silenziosa, area consultazione o area lavoro di gruppo. Il bibliotecario può inoltre visualizzare in ogni momento l'elenco delle prenotazioni effettuate per ciascuna sala e monitorarne lo stato.

Ogni studente autenticato dispone di una propria interfaccia nella quale può consultare le sale disponibili e verificare, per ciascun giorno, le fasce orarie prenotabili. Una prenotazione viene effettuata selezionando una sala, una data, una fascia oraria e, se prevista, una specifica area o una specifica postazione. Il sistema deve impedire che una stessa postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la prenotazione, essa entra nello stato "attiva".

Lo studente può visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate, con le relative informazioni di sala, data, orario e stato. Deve inoltre essere possibile annullare una prenotazione entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la postazione torna automaticamente disponibile per altri utenti.

Nel giorno della prenotazione, lo studente può effettuare il check-in tramite una apposita interfaccia grafica, selezionando la prenotazione attiva e confermando la propria presenza. Per semplicità, si può supporre che il check-in debba essere effettuato entro un intervallo di tempo limitato rispetto all'orario di inizio della prenotazione. Se il check-in non viene effettuato entro tale intervallo, la prenotazione passa automaticamente nello stato "scaduta" e la postazione viene resa nuovamente disponibile.

Il bibliotecario, attraverso la propria interfaccia, può consultare in tempo reale la situazione delle sale, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna sala e per ciascuno studente.

Ogni studente ha accesso a un profilo personale che consente di consultare le prenotazioni future, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati



alla biblioteca. I bibliotecari, attraverso un'interfaccia dedicata, possono monitorare l'andamento complessivo del servizio, visualizzando il tasso di occupazione delle sale, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate tramite check-in.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli studenti della conferma della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema dovrà inoltre garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.

2. Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Il sistema consente la **registrazione** di **utenti** tramite **autenticazione** con credenziali personali. Al momento della **registrazione**, ciascun utente deve specificare il proprio **ruolo**, scegliendo tra **studente** o **bibliotecario**, oltre a fornire **nome**, **cognome**, **email istituzionale** e **numero di matricola** o **codice identificativo interno**.

Ogni **bibliotecario** ha la possibilità di **creare e gestire** una o più sale studio. Per ciascuna **sala**, mediante apposita interfaccia grafica, è possibile specificare un **nome**, una **descrizione**, il **numero totale di postazioni disponibili** e gli **orari di apertura** giornalieri. Ogni **sala** può essere suddivisa opzionalmente in aree differenti, ad esempio **area silenziosa**, **area consultazione** o **area lavoro di gruppo**. Il **bibliotecario** può inoltre **visualizzare in ogni momento l'elenco delle prenotazioni effettuate** per ciascuna **sala** e monitorarne lo stato.

Ogni **studente** autenticato dispone di una propria interfaccia nella quale può **consultare le sale disponibili** e verificare, per ciascun giorno, le **fasce orarie prenotabili**. Una **prenotazione** viene effettuata selezionando una **sala**, una **data**, una **fascia oraria** e, se prevista, una specifica **area** o una specifica **postazione**. Il sistema deve impedire che una stessa **postazione** venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la **prenotazione**, essa entra nello **stato** "attiva".

Lo **studente** può **visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate**, con le relative informazioni di **sala**, **data**, **orario** e **stato**. Deve inoltre essere possibile annullare una **prenotazione** entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la **postazione** torna automaticamente disponibile per altri **utenti**.

Nel giorno della **prenotazione**, lo **studente** può effettuare il check-in tramite una apposita interfaccia grafica, selezionando la **prenotazione** attiva e confermando la propria presenza. Per semplicità, si può supporre che il check-in debba essere effettuato entro un intervallo di **tempo** limitato rispetto all'orario di inizio della **prenotazione**. Se il check-in non viene effettuato entro tale intervallo, la **prenotazione** passa automaticamente nello **stato** "scaduta" e la **postazione** viene resa nuovamente disponibile.

Il **bibliotecario**, attraverso la propria interfaccia, può **consultare in tempo reale la situazione delle sale**, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna **sala** e per ciascuno **studente**.

Ogni **studente** ha accesso a un **profilo personale** che consente di consultare le prenotazioni future, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati alla biblioteca. I bibliotecari, attraverso un'interfaccia dedicata, possono monitorare l'**andamento complessivo del servizio**, visualizzando il **tasso di occupazione delle sale**, il **numero di prenotazioni giornaliere** e il **numero di prenotazioni non confermate tramite check-in**.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un **sistema di notifiche** che avvisi gli studenti della conferma della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema



dovrà inoltre garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.

Note:

- Fascia oraria = orario

- I termini sottolineati indicano i possibili valori dell'attributo tipo della classe Area

LEGENDA:

Classe

Attributo

Funzionalità

Attore

Classe-Attore

2.2 Revisione dei requisiti

1. *Il sistema deve consentire la registrazione degli utenti.*
2. *Il sistema deve offrire una funzionalità di accesso tramite autenticazione con credenziali personali.*
 - *Ambiguità: bisogna capire se il sistema di autenticazione è interno o esterno*
3. *Di ogni Utente si vuole memorizzare nome, cognome, email istituzionale e ruolo (studente o bibliotecario).*
4. *Di ogni Studente si vuole memorizzare il numero di matricola.*
5. *Di ogni Bibliotecario si vuole memorizzare il codice identificativo interno.*
6. *Il sistema deve offrire al Bibliotecario una funzionalità per creare e gestire Sale Studio.*
7. *Di ogni Sala Studio si vuole memorizzare nome, descrizione, numero totale di postazioni disponibili e orari di apertura giornalieri.*
 - *Ambiguità: bisogna capire se le sale sono aperte la domenica e se ci sono giorni di chiusura straordinaria*
8. *Il sistema deve consentire di suddividere opzionalmente ogni Sala Studio in una o più Aree (tipologia: silenziosa, consultazione, lavoro di gruppo ecc...).*
9. *Il sistema deve consentire al Bibliotecario di visualizzare l'elenco delle prenotazioni effettuate per ciascuna Sala Studio e monitorarne lo stato.*
10. *Il sistema deve offrire agli Studenti autenticati una funzionalità per consultare le sale disponibili.*
11. *Il sistema deve permettere allo Studente di verificare le fasce orarie prenotabili per ogni giorno e per ogni sala.*
 - *Ambiguità: bisogna capire se per "Disponibilità" si intende lo stato di apertura o chiusura della Sala Studio oppure la presenza di posti disponibili*
12. *Il sistema deve consentire allo studente di effettuare una Prenotazione selezionando Sala, data e fascia oraria*
 - *Ambiguità: bisogna capire se le fasce orarie sono predefinite dal bibliotecario (es. slot fissi di 2h: 8-10, 10-12...) o lo studente le definisce liberamente*
13. *Il sistema deve consentire di selezionare, se prevista, una specifica Area o Postazione durante la Prenotazione*
 - *Ambiguità: Bisogna capire se le postazioni appartengono a un'area specifica oppure le aree e le postazioni sono due dimensioni indipendenti oppure se quando la sala non è suddivisa in aree, esistono comunque le postazioni numerate; come funziona l'assegnazione della Postazione? Come per i biglietti aerei?*
14. *Il sistema deve impedire che una stessa Postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria.*
15. *Il sistema deve aggiornare automaticamente lo stato della Prenotazione ad "attiva" una volta completata.*
16. *Di ogni Prenotazione si vuole memorizzare Sala Studio, data, fascia oraria e stato.*
17. *Il sistema deve consentire allo Studente di visualizzare, all'interno del Profilo Personale, l'elenco delle proprie prenotazioni.*
18. *Il sistema deve consentire l'annullamento di una Prenotazione entro un limite temporale precedente all'inizio della fascia oraria.*
 - *Ambiguità: bisogna capire come quantificare l'intervallo di tempo*
19. *Il sistema deve rendere la Postazione nuovamente disponibile per altri utenti in caso di annullamento della prenotazione.*
20. *Il sistema deve offrire allo studente una funzionalità per effettuare il check-in tramite interfaccia grafica.*



- *Ambiguità: bisogna capire come avviene tecnicamente il check-in; tramite codice QR generato sulla postazione, geolocalizzazione, o semplice tasto sulla web-app?*
21. *Il sistema deve consentire il check-in solo entro un intervallo di tempo limitato rispetto all'orario di inizio.*
 22. *Il sistema deve aggiornare lo stato della Prenotazione a "scaduta" se in check-in non viene effettuato entro il determinato intervallo.*
 - *Ambiguità: bisogna capire come quantificare l'intervallo di tempo; cosa succede allo scadere dell'intervallo? Ci sono penalità per lo studente?*
 23. *Il sistema deve rendere nuovamente disponibile la Postazione se la Prenotazione risulta "scaduta".*
 24. *Il sistema deve offrire al Bibliotecario una funzionalità per consultare, in tempo reale, lo stato delle Sale Studio (postazioni libere/occupate e prenotazioni attive per la giornata corrente).*
 25. *Il sistema deve offrire al Bibliotecario l'accesso allo storico delle Prenotazioni per Sala Studio e Studente.*
 26. *Il sistema deve consentire allo Studente di consultare le Prenotazioni future, passate, annullate e numero totale di accessi alla biblioteca.*
 27. *Il sistema deve offrire al Bibliotecario una funzionalità per monitorare il tasso di occupazione, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate.*
 28. *Il sistema deve essere accessibile via web sia da desktop che da dispositivi mobili con interfaccia grafica intuitiva e responsiva.*
 29. *Il sistema deve inviare le notifiche per confermare le prenotazioni, gli annullamenti e promemoria orari.*
 - *Ambiguità: Bisogna capire se il sistema di notifiche è interno o esterno e il quando del promemoria*
 30. *Il sistema deve garantire la protezione dei dati personali e la corretta gestione dei permessi di visibilità.*

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Utente	Persona generica che accede al sistema previa autenticazione con credenziali personali	
Credenziali	Dati personali (es. email istituzionale e password) utilizzati per l'autenticazione al sistema	
Studente	Utente registrato con ruolo di studente, identificato da matricola, che può prenotare postazioni nelle sale studio	
Bibliotecario	Utente registrato con ruolo di bibliotecario, identificato da codice identificativo interno, che gestisce le sale studio e monitora le prenotazioni	
Sala Studio	Ambiente fisico gestito da un bibliotecario, caratterizzato da nome, descrizione, numero di postazioni e orari di apertura	Sala



Area	Suddivisione opzionale di una sala studio, caratterizzata da una specifica destinazione d'uso (silenziosa, consultazione, lavoro di gruppo)	
Postazione	Singolo posto a sedere all'interno di una sala studio, eventualmente appartenente a una specifica area, prenotabile da un solo studente per fascia oraria	
Fascia Oraria	Intervallo di tempo prenotabile all'interno degli orari di apertura di una sala	Orario
Prenotazione	Atto di riservare una postazione in una sala, per una data e una fascia oraria specifiche, da parte di uno studente	
Storico Prenotazioni	Archivio delle prenotazioni passate consultabile per sala o per studente	
Stato della Prenotazione	Condizione corrente di una prenotazione: può essere "attiva", "annullata", "scaduta" o "confermata"	
Check-in	Azione con cui lo studente conferma la propria presenza in sala il giorno della prenotazione, entro un intervallo di tempo limitato	
Profilo Personale	Sezione dedicata a ciascun utente in cui consultare le proprie informazioni e, per gli studenti, lo storico delle prenotazioni	
Notifica	Avviso inviato allo studente per confermare prenotazioni, segnalare annullamenti o ricordare l'approssimarsi dell'orario prenotato	

2.4 Classificazione dei requisiti

Classificare i requisiti (funzionali, sui dati, vincoli/altri requisiti), riportando le frasi dei requisiti (revisionati) che li esprimono (ciascuna frase deve esprimere un singolo requisito), numerandoli secondo una codifica (es.: RF01, RF02, RFn per i requisiti funzionali; RD01, RD02, RDm per i requisiti sui dati, RQ01, etc per i requisiti non funzionali di qualità, V01... per eventuali vincoli progettuali). Verificare che i requisiti siano chiari, completi e verificabili.

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RF01	Il sistema deve offrire all'Utente una funzionalità per registrarsi	1
RF02	Il sistema deve offrire al Cliente una funzionalità per accedere	2
RF03	Il sistema deve offrire al Cliente una funzionalità per visualizzare un elenco di auto disponibili per il Noleggio	3
RF04	Il sistema deve offrire al Cliente una funzionalità per prenotare un'auto	6
RF05	Il sistema deve inviare un messaggio di conferma al Cliente, dopo una prenotazione	8



....		
------	-------	--

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RD01	Di ogni Auto si vuole memorizzare modello, cilindrata, costo giornaliero e tipo di patente richiesta per la guida	4
RD02	Di ogni Cliente si vuole memorizzare nome, cognome, patente, con relativa tipologia, e credito disponibile	5
....		

2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010)

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUAL-01	Prestazioni	Il sistema deve mostrare le auto disponibili rapidamente	Tempo di risposta	≤ 2 secondi (95% richieste)	Test di carico con utenti concorrenti
RQ-QUAL-03	Disponibilità	Il sistema deve essere disponibile agli utenti	Uptime mensile	$\geq 99,5\%$	Monitoraggio sistema
...					
RQ-QUAL-08	Usabilità	Un utente deve poter prenotare senza formazione	Tempo completamento	≤ 5 minuti	Test con utenti reali

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
----	-----------	--



V01	Per l'invio della conferma di prenotazione, deve essere disponibile un sistema di messaggistica esterno al sistema	
V02	Per certificare la validità della patente inserita dall'Utente in fase di registrazione, deve essere disponibile il sistema della Motorizzazione Civile	

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Utente
- Cliente
- Gestore

Attori Secondari:

- Servizio di messaggistica
- Sistema Motorizzazione Civile

Casi d'uso:

- **UC1**: Registrazione
- **UC2**: Accesso
- **UC3**: PrenotaAuto
- **UC4**: AggiungiAuto
-
-

Casi d'uso di inclusione:

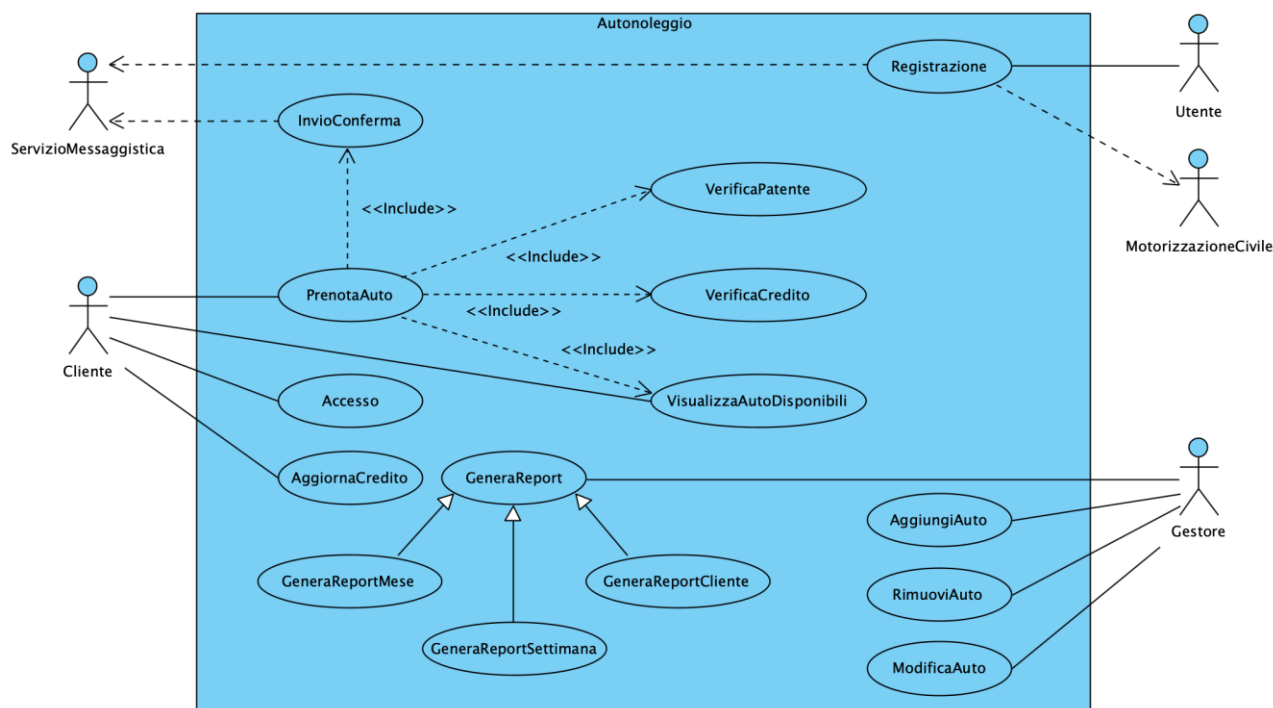
- **UC12**: VisualizzaAutoDisponibili
- **UC13**: VerificaPatente
- **UC14**: VerificaCredito
- **UC15**: InvioConferma

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1 : Registrazione	Utente	MotorizzazioneCivile, ServizioMessaggistica	-	RF01
UC2 : Accesso	Cliente	-	-	RF02
UC3 : PrenotaAuto	Cliente	-	Include VisualizzaAutoDisponibili, VerificaPatente e VerificaCredito	RF03
UC4 : AggiungiAuto	Gestore	-	-	RF10
UC5 : RimuoviAuto	Gestore	-	-	RF11
UC6 : ModificaAuto	Gestore	-	-	RF12
UC7 : GeneraReport	Gestore	-	Generalizzazione di GeneraReportSettimana, GeneraReportMese e GeneraReportCliente	RF13, RF14
UC8 : GeneraReportMese	Gestore	-	-	RF13
UC9 : GeneraReportSettimana	Gestore	-	-	RF13
UC10 : GeneraReportCliente	Gestore	-	-	RF14
UC11 : AggiornaCredito	Cliente	-	-	RF15
UC12 : VisualizzaAutoDisponibili	Cliente	-	Incluso in PrenotaAuto	RF03
UC13 : VerificaPatente	-	-	Incluso in PrenotaAuto	RF06
UC14 : VerificaCredito	-	-	Incluso in PrenotaAuto	RF07, RF08
UC15 : InvioConferma	-	ServizioMessaggistica	Incluso in PrenotaAuto	RF05



2.5.2 Diagramma dei casi d'uso

Riportare il diagramma dei casi d'uso.



2.5.3 Scenari

Selezionare un caso d'uso (dunque una funzionalità) per ogni membro del gruppo, da sviluppare fino alla codifica in Java (dunque, diagramma di sequenza di analisi raffinato, diagramma di sequenza di progettazione, implementazione e test del caso d'uso scelto). Riportare in questa sezione lo scenario principale per il caso d'uso scelto, come nel seguente esempio

Caso d'uso:	PrenotaAuto
Attore primario	Cliente
Attore secondario	-
Descrizione	Un cliente richiede di noleggiare un'auto specificando data e ora di inizio e fine noleggio
Pre-Condizioni	Il Cliente ha effettuato l'accesso
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Cliente accede alla sezione dedicata al noleggio 2. Il Cliente specifica data e ora di inizio e fine noleggio 3. Il sistema mostra le Auto disponibili per il periodo di interesse 4. <<include>> VisualizzaAutoDisponibili 5. if ci sono Auto disponibili 5.1. Il Cliente seleziona l'Auto di suo gradimento 5.2. Il sistema verifica che la patente del Cliente sia idonea alla guida del veicolo di interesse 5.3. <<include>> VerificaPatente 5.4. if le verifiche sono andate a buon fine



	5.4.1. Il sistema verifica che il Cliente abbia la disponibilità sufficiente per il noleggio 5.4.2. <<include>> <i>VerificaCredito</i> 5.4.3. if le verifiche sono andate a buon fine 5.4.3.1. Il sistema procede con la prenotazione 5.4.3.2. Il sistema aggiunge il Noleggio all'archivio dei Noleggi 5.4.3.3. Il sistema invia un messaggio di conferma dell'avvenuta prenotazione al Cliente 5.4.3.4. <<include>> <i>InvioConferma</i>
Post-Condizioni	Il Cliente ha prenotato il noleggio e il sistema valuterà come <i>non disponibile</i> l'Auto selezionata per l'intero periodo di utilizzo
Casi d'uso correlati	<i>VerificaPatente, VerificaCredito, InvioConferma, AggiornaStato</i>
Sequenza di eventi alternativi	Al punto 5, se non ci sono Auto disponibili per il Noleggio, il sistema restituisce un messaggio di errore Al punto 5.4, se le verifiche non sono andate a buon fine, il sistema restituisce un messaggio di errore e chiede al Cliente di selezionare un'altra autovettura Al punto 5.4.3, se le verifiche non sono andate a buon fine, il sistema restituisce un messaggio di errore e chiede al Cliente di selezionare un'altra autovettura, ritornando al punto 3, o di aggiornare la propria disponibilità

2.6 Diagramma delle classi

Sviluppare il Diagramma delle classi di analisi corrispondente al System Domain Model. Realizzare anche una versione raffinata che includa le responsabilità delle classi.

A seguire, una tabella che riassume alcune responsabilità:

RESPONSABILITÀ	CLASSE
<i>Registrazione</i>	Autonoleggio
<i>Accesso</i>	Autonoleggio
<i>PrenotaAuto</i>	Cliente
<i>AggiungiAuto</i>	Autonoleggio
<i>RimuoviAuto</i>	Autonoleggio
<i>ModificaAuto</i>	Auto
-	-
-	-
-	-
-	-
-	--
<i>VerificaCredito</i>	Cliente
-	-
<i>AggiornaCredito</i>	Cliente

[Qui va aggiunta una descrizione/giustificazione di come è stata assegnata la responsabilità, tenendo presente anche i pattern GRASP.]



Registrazione e Accesso sono responsabilità di **Autonoleggio**, in quanto <<information expert>> di Clienti.

PrenotaAuto è responsabilità di **Cliente**, perché è <<Creator>> della classe Noleggio.

.....



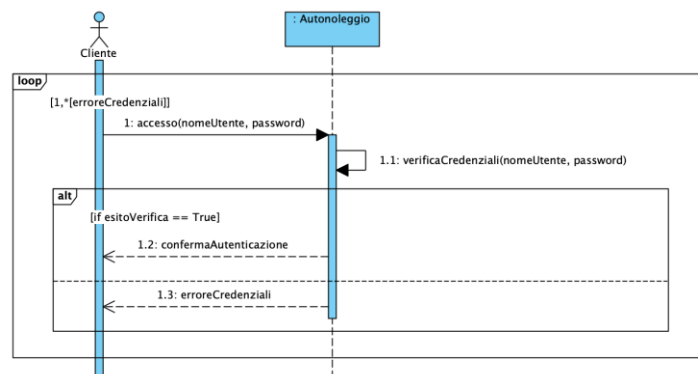
2.7 Diagrammi di sequenza

Riportare il diagramma di sequenza di analisi per le funzionalità (ossia i caso d'uso) da implementare (sceglierne una non banale per ogni membro del gruppo) e che saranno sviluppate fino alla codifica in Java ed al test.

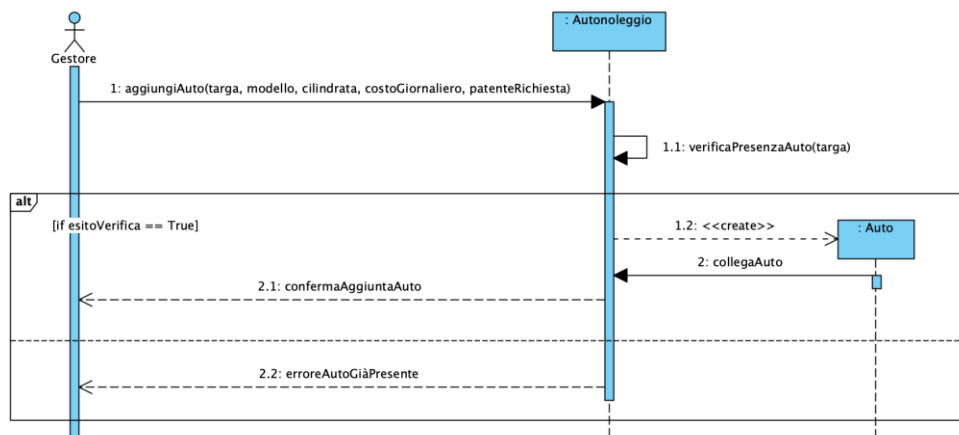
Si riportano alcuni esempi di diagrammi di sequenza di analisi per i casi d'uso precedentemente individuati.

2.7.1 Accesso

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *Accesso*, ha fatto sorgere la necessità di definire un metodo, specifico per la classe **Autonoleggio**, **verificaCredenziali(nomeUtente, password)**, privato, per consentire all'autonoleggio di verificare che le credenziali inserite dall'utente siano valide.



2.7.2 AggiungiAuto



Per le stesse ragioni, è stato necessario inserire all'interno della classe **Autonoleggio** il metodo privato **verificaPresenzaAuto(targa)**, col fine ultimo di individuare se l'auto che il gestore intende aggiungere è già presente all'interno del parco auto.



2.8 Diagramma delle classi raffinato

Le aggiunte e le modifiche fatte nel corso della costruzione dei Sequence Diagrams hanno determinato lo sviluppo di un [Diagramma delle Classi](#) raffinato che riporta maggiori dettagli sugli attributi e le principali operazioni delle classi:

Riportare CD raffinato

3. Piano di test funzionale

Progettare i casi di test funzionale con la tecnica del *Category Partition Testing*. Descrivere il procedimento di calcolo.

Si intende progettare i casi di test funzionale con la tecnica del *Category Partition Testing*.

3.1 Registrazione Utente

REGISTRAZIONE				
NOME	COGNOME	PATENTE	TIPOPATENTE	CREDITO
- Stringa di caratteri di lunghezza ≤ 40 - Stringa di caratteri di lunghezza > 40 [ERROR] - Stringa che contiene simboli che non sono caratteri [ERROR]	- Stringa di caratteri di lunghezza ≤ 40 - Stringa di caratteri di lunghezza > 40 [ERROR] - Stringa che contiene simboli che non sono caratteri [ERROR]	- Stringa di caratteri alfanumerici di lunghezza $= 10$ - Stringa di caratteri alfanumerici di lunghezza > 10 [ERROR] - Stringa di caratteri alfanumerici di lunghezza < 10 [ERROR] - Stringa contenente caratteri speciali [ERROR]	- Stringa di caratteri alfanumerici di lunghezza ≤ 3 - Stringa di caratteri alfanumerici di lunghezza > 3 [ERROR] - Stringa contenente caratteri speciali [ERROR]	- Numero decimale ≥ 0 - Numero decimale < 0 [ERROR] - Numero che contiene simboli che non sono numeri [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $3 * 3 * 4 * 3 * 3 = 324$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 11 (2 per Nome, 2 per Cognome, 3 per Patente, 2 per TipoPatente, 2 per Credito).

Il numero di test risultante è: $(1*1*1*1*1) + 11 = 12$.



TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Nome valido Cognome valido Patente valida TipoPatente valido Credito valido		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Ciente registrato	Si ricevono le credenziali di accesso per email
2	Nome stringa > 40 caratteri	Nome stringa > 40 caratteri [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Marioooooooooooooooooooooo ooooooooooooooooooooooooooooo ooooooooooooooooooooooooooooo", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Nome troppo lungo!	
3	Nome stringa con simboli	Nome stringa con simboli [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Ma-r.o", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Formato Nome errato!	
4	Cognome stringa > 40 caratteri	Nome valido, Cognome stringa > 40 caratteri [ERROR], Patente, TipoPatente, Credito validi		{Nome: "Mario", Cognome: "Rossiiiiiiiiiiiiiiiiiiiiiiiiii iiii iiiiiiiiiiiiiiiiiiiiiiiiiiiii iiiiiiiiiiiiiiiiiiiiiiiiiiiiiii", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Cognome troppo lungo!	
5	Cognome stringa con simboli	Nome valido, Cognome stringa con simboli [ERROR], Patente, TipoPatente, Credito validi		{Nome: "Mario", Cognome: "R!si-", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Formato Cognome errato!	
6	Patente stringa > 10 caratteri	Nome, Cognome validi, Patente stringa > 10 caratteri [ERROR], TipoPatente, Credito validi		{Nome: "Mario", Cognome: "Rossi", Patente: "NA80830D023930DL23", TipoPatente: "B1", Credito "230.50"}	Patente troppo lunga!	
7	Patente stringa < 10 caratteri	Nome, Cognome validi, Patente stringa < 10 caratteri [ERROR], TipoPatente, Credito validi		{Nome: "Mario", Cognome: "Rossi", Patente: "NA80LP", TipoPatente: "B1", Credito "230.50"}	Patente troppo breve!	
8	Patente stringa con caratteri speciali	Nome, Cognome validi, Patente stringa con caratteri speciali [ERROR], TipoPatente, Credito validi		{Nome: "Mario", Cognome: "Rossi", Patente: "NA.68@9!3-", TipoPatente: "B1", Credito "230.50"}	Formato Patente errato!	

4. Progettazione

4.1 Diagramma delle classi

Riportare il diagramma delle classi di progettazione che rispetti il modello architetturale del BCED (senza violarne le regole di dipendenza fra i livelli). Il class diagram dovrà includere le classi Entity (corrispondenti a quelle di dominio), nonché le Classi Boundary, Controller, e Dati (ossia le classi DAO o Wrapper per l'accesso al Database)

Il diagramma dovrà essere organizzato utilizzando i package per raggruppare le classi dello stesso layer.

In questo diagramma saranno inoltre documentate le scelte di progetto fatte, come ad esempio:

- Reificare eventuali classi associative del diagramma delle classi di analisi.
- Specificare argomenti e tipo di ritorno delle operazioni (per quelle più significative, coinvolte nei casi d'uso sviluppati fino alla implementazione).
- Decidere i versi di navigabilità delle associazioni

4.1.1 Traduzione classi ed associazioni

Spiegare le scelte effettuate

4.1.2 Pattern BCED

4.1.2.1 Package Boundary

Il package Boundary contiene tutti gli oggetti responsabili dell'interfaccia utente e della logica di presentazione; a questo livello tutte le classi corrispondono a delle interfacce e i relativi attributi non sono altro che gli elementi che le compongono, visualizzati a video.

Riportare il Package Boundary

4.1.2.2 Package Controller

Questo package contiene gli oggetti che percepiscono gli eventi generati dalle interazioni con l'interfaccia utente e ne demandano la gestione all'unico componente del sistema software responsabile della gestione della Business Logic, il package Entity.

Riportare il Package Controller



4.1.2.3 Package Entity

Il Package Entity contiene tutti gli oggetti che rappresentano la semantica delle entità del dominio applicativo e corrispondono alle strutture dati presenti all'interno del database di persistenza.

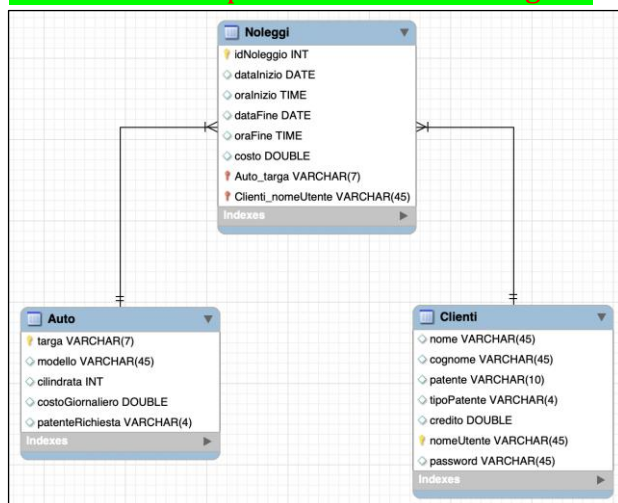
Riportare il Package Entity

4.1.2.4 Package Database

Riportare le scelte progettuali adottate per il database di persistenza

(ossia il Modello E-R costruito ad esempio con l'editor di MySQL Workbench).

Si veda ad esempio lo schema E-R allegato:



Il Package Database contiene tutte le classi responsabili dell'estrazione dei dati dal DB, esponendo una vera e propria interfaccia che di fatto rende indipendenti le classi della Business Logic (Entity) dalla tecnologia di persistenza utilizzata.

In particolare, tra le strategie di risoluzione del problema dell'**impedance mismatch**, che nasce dalla mancata corrispondenza tra il modello Object Oriented e quello relazionale, si è deciso di adottare quella delle classi **DAO** (Data Access Objects), che consiste nell'utilizzo di appositi oggetti per l'accesso ai dati.

Ognuna di queste classi conterrà i metodi CRUD per l'interrogazione e la manipolazione della corrispondente classe di dominio (*query*), implementati in funzione di un'ulteriore classe,



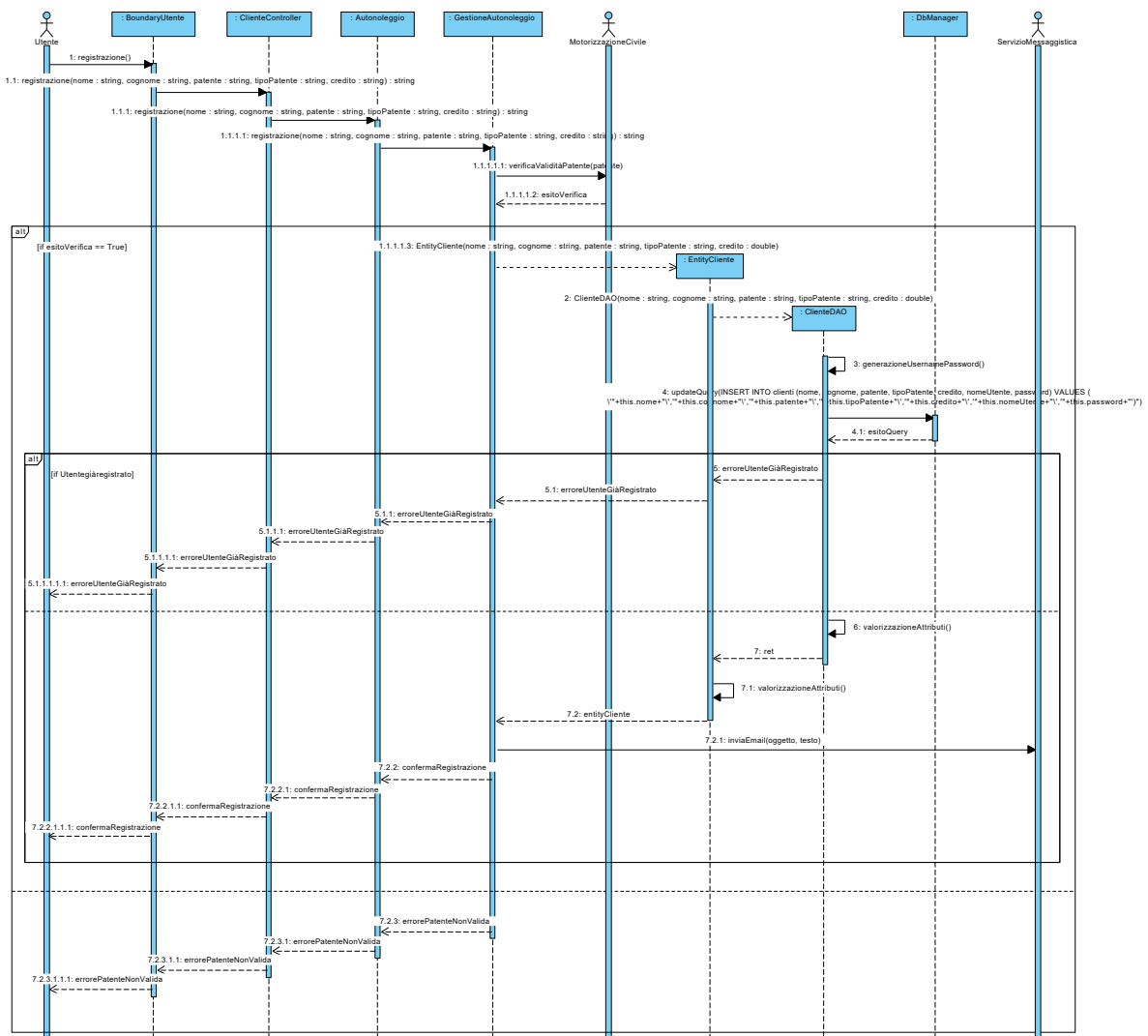
DBManager, che costituisce di fatto l'unico punto di accesso vero e proprio al DB, sfruttando i metodi che questa mette a disposizione.

Riportare il Package Database

4.2 Diagrammi di sequenza

Si riportino di seguito i diagrammi di sequenza di progetto per il/i casi d'uso sviluppati fino alla codifica in Java.

4.2.1 Registrazione



I sequence progettati sono stati fondamentali per la corretta implementazione dell'applicazione software ed ha fatto nascere la necessità di definire ulteriori classi, metodi e funzioni, che hanno arricchito passo dopo passo il [Diagramma delle Classi di Progettazione](#), fino ad ottenere la seguente versione finale:

Riportare il Diagramma delle Classi di Progettazione di Dettaglio

5. Implementazione

Non includere il codice sorgente, ma descrivere l'implementazione in Java, descrivendo gli artefatti di codifica:

- Elencare:
 - o package, classi, tipi di eccezione definiti
- Elencare gli artefatti necessari per l'installazione ed esecuzione del programma, senza ovviamente l'ambiente di sviluppo come Eclipse (DB h2, eventuali librerie e versioni di Java che l'utilizzatore deve avere installati, file .class, .jar, ...)
- Produrre un eventuale diagramma di deployment
- Eventualmente inserire la documentazione del codice prodotta con Javadoc (relativamente alle funzionalità implementate)

5.1 Package Database

TBD

5.2 Package Entity

TBD

5.3 Package Controller

TBD

5.4 Package Boundary

TBD

5.5 Package DTO

L'introduzione di tale package, estraneo al pattern BCED, nasce dall'esigenza di mostrare sulla GUI collezioni di elementi.

Da questo punto di vista, il problema principale è proprio quello che, qualora una determinata chiamata a funzione restituisse alla GUI un elenco di entity, questa, per poterlo visualizzare



correttamente a video, dovrebbe conoscere di fatto la struttura interna di tale classe Entity, ma ciò porterebbe con sé un accoppiamento troppo elevato e quindi una chiara violazione dei vincoli del pattern a livelli adottato.

Si introduce allora il concetto di **Data Transfer Object (DTO)**, un oggetto in grado di trasportare dati tra processi (nel caso in oggetto tra livelli). Le classi DTO hanno tipicamente una struttura che rispecchia quella dell'entity di cui vanno a supporto, in particolare gli attributi coincidono con quelli dell'entity che si intendono visualizzare a schermo.

5.6 Diagramma di Deployment

I diagrammi di deployment sono utilizzati per mostrare l'architettura fisica del sistema software realizzato; sono particolarmente utili per valutare, durante lo sviluppo, come un'applicazione si distribuisce tra le varie macchine.

- .

6. Testing

6.1 Test Strutturale

Costruire il Control Flow Graph per uno o due dei metodi delle classi implementate (si scelgano metodi non proprio banali), e:

- si mostri il calcolo del numero cicломatico;
 - si indichino i percorsi linearmente indipendenti;
 - si progettino i casi di test per coprire i percorsi individuati.
- Prima o a fianco del CFG riportare il codice Java del metodo.

6.1.1 Complessità ciclomatica

Si intende costruire il Control Flow Graph per due dei metodi delle classi implementate e mostrare il calcolo del numero cicломatico e i percorsi linearmente indipendenti.

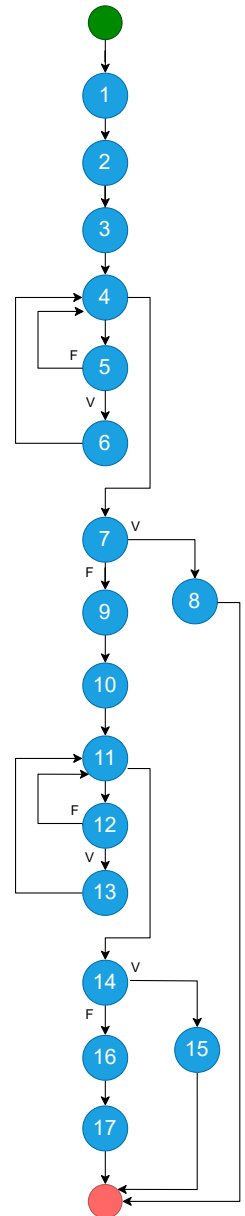
6.1.1.1 *inserisciAutoModifiche – GestioneParcoAuto*

```
public String inserisciAutoModifiche(String targa, String modello, String cilindrata, String costoGiornaliero, String patenteRichiesta) {
```

```

EntityElencoAuto elenco = new EntityElencoAuto(); (1)
ArrayList<AutoDTO> elencoAuto = elenco.getListaAuto(); (2)
boolean controlloTarga = false; (3)
for(int i=0; i<elencoAuto.size(); i++) { (4)
    if(targa.compareTo(elencoAuto.get(i).getTarga())==0) { (5)
        controlloTarga = true; (6)
    }
}
if(!controlloTarga) { (7)
    return "Errore, l'auto selezionata è inesistente!"; (8)
}
ArrayList<AutoDTO> elencoAutoDeposito = visualizzaElencoAuto(); (9)
controlloTarga = false; (10)
for(int i=0; i<elencoAutoDeposito.size(); i++) { (11)
    if(targa.compareTo(elencoAutoDeposito.get(i).getTarga())==0) { (12)
        controlloTarga = true; (13)
    }
}
if(!controlloTarga) { (14)
    return "Errore, l'auto selezionata non rientra tra quelle in deposito!"; (15)
}
EntityAuto autoDaModificare = new EntityAuto (targa); (16)
return autoDaModificare.modificaAuto(modello, cilindrata, costoGiornaliero,
patenteRichiesta); (17)
}

```



NUMERO CICLOMATICO:

- numero di regioni chiuse del grafo + 1 = 7
- numero di nodi predicati (4,5,7,11,12,14) + 1 = 7
- # archi - # nodi + 2 = (22 - 17) + 2 = 7

CAMMINI:

- 1) 1-2-3-4-7-8
- 2) 1-2-3-4-5-4-7-8
- 3) 1-2-3-4-5-6-4-7-8
- 4) 1-2-3-4-7-9-10-11-14-15
- 5) 1-2-3-4-7-9-10-11-12-11-14-15
- 6) 1-2-3-4-7-9-10-11-12-13-11-14-15
- 7) 1-2-3-4-7-9-10-11-14-16-17

CASI di TEST

TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/FAIL)
TC_1	1-2-3-4-7-8	Percorso in cui l'auto da modificare non esiste nel sistema	La condizione al punto 7 del CFG è true	targa= CC123EE, modello=PANDA, cilindrata=800, costoGiornaliero=30, patenteRichiesta=B	La targa CC123EE non è memorizzata nel sistema	Errore, l'auto selezionata è inesistente!		PASS/FAIL
2	1-2-3-4-7-9-10-11-14-15	Percorso in cui l'auto da modificare esiste nel	La condizione al punto 7 del CFG è false, La	targa= CC123EE, modello=PANDA, cilindrata=800, costoGiornaliero=30, patenteRichiesta=B	La targa CC123EE è memorizzata nel sistema, ma non è	Errore, l'auto selezionata non rientra tra quelle	...	PASS/FAIL



		sistema, ma non esiste nel deposito	condizione al punto 14 del CFG è true		presente in deposito	in deposito!		
..								

6.2 JUnit – Test di Unità

Riportare a scopo esemplificativo alcuni casi di utilizzo di **JUnit** per testare alcune classi del progetto, il framework di testing di unità automatizzato per il linguaggio di programmazione Java.

6.3 Test funzionale

Segue una descrizione in forma tabellare dei risultati dell'esecuzione dei test funzionali precedentemente pianificati.

REGISTRAZIONE									
TEST SUITE									
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese	Output ottenuti	Post-condizioni ottenute	Esito (FAIL, PASS)
1	Tutti input validi	Nome valido Cognome valido Patente valida TipoPatente valido Credito valido		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Cliente registrato	Si ricevono le credenziali di accesso per email	Registrazione avvenuta con successo, controlla la mail per visualizzare le credenziali di accesso	Il Cliente si è registrato e ha ottenuto l'accesso alla piattaforma	PASS
2	Nome stringa > 40 caratteri	Nome stringa > 40 caratteri [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Marioooooooooooooooooooooooooooooo oooooooooooooooooooooooooooooooooooo", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Nome troppo lungo!		Errore, il nome inserito è troppo lungo!		PASS
3	Nome stringa con simboli	Nome stringa con simboli [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Ma-r.o", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Formato Nome errato!		Errore, il formato del nome inserito non è valido!		PASS
									
12	Credito numero con simboli	Nome, Cognome, Patente, TipoPatente validi, Credito numero con simboli [ERROR]		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "7@.1!"}	Formato Credito errato!		Errore, il formato del credito inserito non è valido!		PASS